

Explorando o valor da autoavaliação do aluno em cursos introdutórios de programação.

Rodrigo Duran
Universidade Aalto
Finlândia

rodrigo.duran@aalto.fi

Jan-Mikael Rybicki
Universidade Aalto
Finlândia

jrybicki@cc.hut.fi

Juha Sorva
Universidade Aalto
Finlândia

juha.sorva@aalto.fi

Arto Hellas
Universidade de Helsinque
Finlândia

arto.hellas@helsinki.fi

RESUMO

Professores de programação têm grande necessidade de instrumentos fáceis de usar que forneçam informações confiáveis e pedagogicamente úteis sobre a aprendizagem dos alunos. Atualmente, não existem ferramentas validadas para avaliar rapidamente a compreensão dos alunos sobre conhecimentos básicos de programação. Os inventários de conceitos e o questionário SCS1 podem oferecer grandes benefícios; este artigo explora o valor adicional que pode ser obtido com métricas de autoavaliação relativamente simples. Aplicamos um instrumento de autoavaliação simplificado (SEI) em um curso introdutório de programação e comparamos os resultados com medidas de desempenho existentes, como notas de exames e o SCS1. Constatamos que o SEI apresenta uma correlação semelhante com um exame de redação de programas, assim como o SCS1, embora ambos os instrumentos apresentem correlação apenas moderada com o exame e entre si. Além disso, os alunos são muito mais propensos a responder voluntariamente ao SEI simplificado do que ao SCS1. No geral, nossos resultados sugerem que tanto o SEI quanto outros instrumentos precisam ser aprimorados significativamente e delineiam trabalhos futuros nesse sentido.

CONCEITOS CCS

• **Temas sociais e profissionais** e **Educação em ciência da computação**;
Currículos modelo; Avaliação do aluno.

PALAVRAS-CHAVE

Autoavaliação, avaliação, CS1, programação introdutória

Formato de referência da ACM:

Rodrigo Duran, Jan-Mikael Rybicki, Juha Sorva e Arto Hellas. 2019. Explorando o valor da autoavaliação do aluno na programação introdutória. Em International Computing Education Research Conference (ICER '19), 12 a 14 de agosto de 2019, Toronto, ON, Canadá. ACM, Nova York, NY, EUA, 10 páginas. <https://doi.org/10.1145/3291279.3339407>

1 INTRODUÇÃO

Os professores avaliam os alunos por diversos motivos. Por exemplo, desejam fornecer feedback, avaliar seu próprio ensino, considerar o conhecimento prévio da turma no planejamento do curso [18, 42, 46, 52, 56, 57], recomendar atividades ou módulos de aprendizagem adequados para alunos individualmente [7], ajudar os alunos a se autoavaliarem e a autorregular sua aprendizagem.

[60], identificar alunos em risco [54] e comparar alunos com benchmarks interinstitucionais [11, 38].

Essas ambições criam a necessidade de uma variedade de avaliações. Às vezes, uma avaliação detalhada é necessária; outras vezes, uma nota numérica geral é suficiente. Em alguns casos, é aceitável que os alunos autoavaliem suas habilidades; em outros, é necessário um avaliador externo ou um teste formal. Embora certas avaliações devam ser obrigatórias, outras devem permanecer voluntárias. Em alguns casos, é aceitável que a avaliação leve tempo; em outros, a avaliação deve ser rápida, mesmo que isso comprometa a precisão.

Em algumas áreas da educação, ferramentas de avaliação validadas estão disponíveis para uso dos professores, como o Quadro Europeu Comum de Referência para Línguas (CEFR) [11], enquanto a educação em computação ainda carece de tais ferramentas. Apesar disso, a avaliação das habilidades em computação tem atraído muito interesse. Uma linha de pesquisa anterior visa construir inventários de conceitos e testes similares para avaliar a capacidade e o conhecimento de programação dos alunos. O questionário SCS1 [35], por exemplo, utiliza um conjunto de 27 questões de múltipla escolha sobre programas em pseudocódigo para medir o conhecimento dos alunos sobre programação imperativa. Tais instrumentos, que visam diretamente o conhecimento do aluno, são potencialmente muito valiosos. No entanto, realizar um teste desse tipo consome muito tempo e pode ser estressante para os alunos; além disso, testes complexos são pouco adequados para avaliações repetidas dentro de um mesmo módulo de estudo.

Outros pesquisadores avaliaram indiretamente as habilidades computacionais dos alunos através da lente da autoeficácia: a crença dos alunos em sua capacidade de realizar determinadas tarefas. Danielsiek et al. [12] validaram um instrumento para medir a autoeficácia em cursos de algoritmos. A autoeficácia demonstrou ter uma forte correlação com o sucesso na aprendizagem [4, 24, 40] e pode ser avaliada rapidamente. No entanto, não é uma medida direta da compreensão conceitual dos alunos.

Para avaliar o conhecimento prévio, as habilidades e a compreensão conceitual dos alunos, eles podem realizar uma autoavaliação de suas próprias habilidades. A autoavaliação consiste na avaliação ou julgamento do "valor" do próprio desempenho e na identificação de pontos fortes e fracos com o objetivo de melhorar os resultados da aprendizagem [38]. Em suma, a autoavaliação é particularmente adequada para avaliar trabalhos passados ou atuais, e a autoeficácia busca estimar a confiança e extrapolar o desempenho futuro.

Ross [38] revisa evidências de pesquisas anteriores que sugerem que a autoavaliação contribui para um maior desempenho acadêmico e melhor comportamento dos alunos. Além disso, há evidências que comprovam a confiabilidade da autoavaliação em termos de consistência interna, consistência entre tarefas e itens e em curtos períodos de tempo. No entanto, idade, experiência, expectativa de avaliação e desempenho anterior influenciam a forma como os alunos se autoavaliam. Assim,

É concedida permissão para fazer cópias digitais ou impressas, totais ou parciais, desta obra para uso pessoal ou em sala de aula, sem custo, desde que as cópias não sejam feitas ou distribuídas para fins lucrativos ou comerciais e que as cópias contenham este aviso e a citação completa na primeira página. Os direitos autorais dos componentes desta obra que pertencem a terceiros que não o (s) autor(es) devem ser respeitados. É permitido fazer resumos com a devida atribuição. Para copiar, republicar, postar em servidores ou redistribuir para listas, é necessária permissão prévia específica e/ou o pagamento de uma taxa. Solicite permissões pelo e-mail permissions@acm.org. ICER '19, 12 a 14 de agosto de 2019, Toronto, ON, Canadá © 2019 Direitos autorais reservados ao(s) proprietário(s)/autor(es). Direitos de publicação licenciados à ACM. ACM ISBN 978-1-4503-6185-9/19/08...\$15.00 <https://doi.org/10.1145/3291279.3339407>

As autoavaliações tendem a apresentar correlação apenas moderada com as avaliações dos professores. Miller [30] demonstra que os efeitos de teto na autoavaliação estudos estão relacionados a pontuações excessivamente vagas ou difíceis de entender. critérios. A sensibilidade do instrumento pode ser aumentada adicionando níveis de medição e critérios precisamente definidos.

A autoavaliação também é um elemento-chave da aprendizagem autorregulada [22] e está intimamente relacionada à autoeficácia. Na literatura, os termos Autoavaliação e autoexame são frequentemente usados como sinônimos; Usaremos o termo autoavaliação ao discutimos nosso trabalho.

Este artigo dá continuidade ao trabalho de Duran et al. [13] para desenvolver e validar um instrumento de autoavaliação (IAA). O IAA complementa as ferramentas de avaliação existentes, realizando uma pesquisa rápida com os alunos. Compreensão dos conceitos básicos de programação.

Pretendemos que o SEI inclua as seguintes características: (1) Pode ser respondida rapidamente. Os alunos se voluntariam para responder ao SEI. (1) Quando solicitado e complete-o; (2) Captura informações pedagogicamente úteis informações sobre o conhecimento prévio de programação dos alunos e o crescimento desse conhecimento à medida que aprendem, refletindo os alunos progresso ao longo de uma trajetória de aprendizagem de habilidades de compreensão de código; (3) Destaca as diferenças entre os conceitos de programação (em vez de do que ser uma única medida genérica de habilidade de programação ou autoconfiança); (4) Apesar de não ser um teste rigoroso de habilidade, Fornece algumas informações sobre o desempenho real conforme medido.

por instrumentos mais pesados; (5) Não parece um exame", mas promove a reflexão e a autoavaliação; (6) É fácil Para que os alunos compreendam sem treinamento prévio: os alunos precisam Não responda a perguntas sobre conceitos altamente abstratos e genéricos. objetivos de aprendizagem; (7) Em vez disso, é formulado em termos de aprendizagem objetivos, linguagem de programação e outros fatores contextuais de um curso CS1 específico; (8) Sua estrutura genérica pode ser adaptada a diferentes cursos; (9) É fácil de administrar em grande escala.

Na prática, nosso SEI é uma pesquisa online que pede aos alunos que avaliem sua própria compreensão de conceitos de programação selecionados. Por Ao aplicar o SEI repetidamente em diferentes etapas de um MOOC introdutório de programação, exploramos a praticidade do SEI.

Além disso, ao investigar o SEI, também administramos o SCS1 no final do MOOC e verificou se nosso aluno

A coorte é comparável em termos de desempenho no SCS1 a outras coortes. relatado na literatura. Isso nos deu a oportunidade de replicar Alguns dos resultados do incipiente conjunto de pesquisas sobre o SCS1. Nossas questões de pesquisa são as seguintes:

- RQ1** Como os alunos respondem ao SEI e ao SCS1: eles se voluntariam? Para respondê-las e completá-las?
- RQ2** O SEI captura a crescente confiança dos alunos do CS1 em seus conhecimentos? conhecimento de programação à medida que aprendem?
- RQ3** O SEI revela diferenças conceituais específicas nos alunos? avaliações de suas habilidades?
- RQ4** O SEI possui validade externa: em que medida os alunos As autoavaliações estão de acordo com as métricas de desempenho existentes, tais como: como notas de exame e o SCS1?
- RQ5** Nossos alunos acham as mesmas questões SCS1 difíceis que os outros? O que fizeram os alunos em estudos anteriores?

2 TRABALHOS RELACIONADOS

A comunidade de pesquisa em educação computacional (CER, na sigla em inglês) introduziu inúmeros instrumentos para avaliar o conhecimento dos alunos.

e habilidades (ex: [2, 12, 16, 17, 21, 27, 31, 33, 36, 37, 39, 41, 45, 55]). Margulieux et al. [28] fornecem uma lista abrangente dos validados nesta seção, comentamos estudos selecionados que ou fornecem parâmetros de referência úteis ou são semelhantes ao presente trabalho de Explorando a autoavaliação ou a autoeficácia em um contexto computacional.

Alguns dos testes validados mais conhecidos de conhecimento em programação incluem o FCS1 [49] e o SCS1 [6, 35]. O FCS1 é um teste de múltipla escolha sobre conceitos comumente abordados em cursos de CS1: os alunos responder a perguntas sobre programas escritos em pseudocódigo imperativo . Foi validado e demonstrou ser confiável e positivamente correlacionado com as notas do curso. O SCS1 [35] é mais facilmente disponível versão isomórfica do FCS1; possui consistência interna semelhante. e uma forte correlação com o FCS1. Desde a sua introdução, O FCS1 e o SCS1 foram usados em muitos outros estudos como benchmarks de desempenho [23, 32, 50, 51, 58].

Danielsiek et al. [12] desenvolveram e validaram um questionário Este instrumento foi desenvolvido para avaliar a autoeficácia em um curso de algoritmos. O instrumento incentiva os alunos a se autoavaliarem, classificando 21 afirmações curtas sobre suas habilidades relacionadas a algoritmos. Os autores o aplicaram diversas vezes, demonstrando que a autoeficácia dos alunos aumentou à medida que aprendiam.

Murphy e Tenenberg [31] examinaram estudantes de nível superior de Ciência da Computação capacidade de autoavaliar seu conhecimento de estruturas de dados. Eles compararam uma única questão que pedia aos alunos que autoavaliassem seu desempenho em um teste com a nota real que obtiveram no teste. Os resultados revelou uma correlação moderada entre as estimativas dos alunos (n=61). e desempenho em testes; alunos com pontuações mais altas apresentaram maior precisão. Em outro estudo, Kallia e Sentance [21] utilizaram uma combinação de duas medidas de autoeficácia e uma única questão de autoavaliação Investigar as relações entre concepções errôneas relacionadas à função e a autoeficácia para programação. Eles descobriram que os alunos

Alunos com ideias equivocadas sobre programação demonstram níveis significativamente mais baixos de autoeficácia do que alunos com poucas ideias equivocadas. ou não têm ideias erradas e superestimam seu desempenho no auto-avaliação.

Ngai et al. [33] procuraram aliviar a avaliação dos alunos de CS1 ansiedade, ensinando-os a se autoavaliarem. Os alunos do Um estudo piloto (n=13) utilizou uma rubrica específica para que os participantes se autoavaliassem em três aspectos. habilidades de programação — depuração, codificação e programação — e até atribuíram a si mesmos uma nota geral. Ngai et al. descobriram que o As autoavaliações apresentaram forte correlação com as notas atribuídas pelos professores. (.78) e as notas de um exame final (.85).

Alaoutinen e Smolander [2] criaram um instrumento de pesquisa destinado a professores para avaliar o conhecimento dos alunos, bem como para Os alunos devem autoavaliar seu desenvolvimento. Com base na Taxonomia de Bloom [3], o instrumento pede aos alunos que avaliem seus conhecimentos e habilidades no contexto da programação introdutória no Linguagem C. Os alunos deveriam estimar em que nível eles conseguiriam. operar com sucesso, selecionando um nível de Bloom diferente para cada conceito . As autoavaliações dos alunos apresentaram forte correlação com o exame. notas (0,54) e os tópicos considerados mais difíceis pelos professores também foram consideradas mais difíceis pelos alunos.

Embora muitos dos estudos acima coincidam com o nosso em um aspecto ou Outra, nenhuma aborda a mesma combinação de objetivos que nós. O estudo de Alaoutinen e Smolander [2] é provavelmente o mais semelhante ao nosso; nosso trabalho pode ser visto como um esforço para aprimorar o deles. Comparado Para o estudo deles, pretendemos (1) formular nosso SEI em termos mais concretos.

(como sugerido pela literatura sobre autoavaliação), aliviando assim os alunos do fardo de interpretar os níveis de Bloom, que é sabidamente difícil até mesmo para os professores [29]; e (2) conectar nosso SEI a uma trajetória de aprendizagem de habilidades de compreensão de código.

A próxima seção resume o desenvolvimento e a estrutura do nosso SEI, conforme apresentado originalmente por Duran et al. [13]. As seções subsequentes apresentam o desenho do nosso estudo e respondem às cinco questões de pesquisa listadas acima.

3. DESENVOLVIMENTO DO INSTRUMENTO

Nosso protótipo SEI (disponível em <https://goo.gl/nGR9Th>) destina-se à programação orientada a objetos imperativa em Java, mas pode ser adaptado a outros contextos. O SEI é inspirado na rubrica de autoavaliação do Quadro Europeu Comum de Referência para Línguas (CEFR) [11, p. 26-27] e nos trabalhos teóricos e empíricos em CER que sugerem uma hierarquia de habilidades de compreensão de código [9, 14, 25, 26, 47, 48, 59]. Os níveis da estrutura são organizados em uma hierarquia de habilidades, desde a compreensão do significado, sintaxe e semântica de um conceito (A), até a previsão do efeito da sintaxe no comportamento do programa, ou seja, rastreamento (B), e a capacidade de compreender e resumir padrões (C). Introduzimos granularidade a esses níveis adicionando subníveis de complexidade, como o número e a qualidade dos elementos que interagem [5, 14, 44] e a familiaridade dos alunos com o código.

Os designers instrucionais podem usar quantos níveis forem necessários em seu instrumento adaptado. Cada subnível inclui uma declaração que reflete o nível conceitual de compreensão do código e a habilidade associada, incluindo o nível de complexidade.

Como prevíamos que os alunos teriam formações e níveis de conhecimento variados, introduzimos três subníveis ao primeiro nível (A): um nível de desconhecimento (A0), um nível de familiaridade com o significado geral de um conceito (A1) e um nível de compreensão da sintaxe e da semântica de um conceito (A2). Ao dominar o nível A, os alunos conseguem compreender conceitualmente um conceito (por exemplo, um laço) e reconhecer sua sintaxe no código (por exemplo, a palavra-chave `for`), sem, no entanto, serem capazes de prever sua funcionalidade quando aplicada a um caso concreto.

Após dominar o nível A, os alunos podem rastrear código no nível B. Alunos no nível B1 conseguem rastrear código com poucos elementos, prevendo a saída do código ao usar valores concretos e código com convenções de nomenclatura descritivas (por exemplo, o código "for count in range of 10" denota "repetir 10 vezes"). No nível B2, os alunos podem alcançar os mesmos resultados do nível B1, mas também conseguem lidar com vários elementos interagindo simultaneamente (por exemplo, código incluindo muitos loops adjacentes com estruturas condicionais internas). Reconhecer padrões e resumir código são as habilidades de compreensão de código mais avançadas. No nível C1, os alunos conseguem reconhecer padrões usando um conceito, mesmo que as convenções de nomenclatura do código não sejam padronizadas, e compreendem códigos com uma variedade de entradas abstratas (por exemplo, reconhecem padrões de baixo nível, como percorrer uma coleção, verificar números negativos ou adicionar elementos a uma lista). Finalmente, no nível C2, os alunos devem dominar todos os outros níveis e conseguem generalizar a finalidade do código e resumir-la em linguagem simples (por exemplo, resumir o código como "uma função de filtro que recebe uma lista como entrada e retorna todos os números positivos").

Avaiamos conceitos de programação tipicamente encontrados em cursos introdutórios de Ciência da Computação: variáveis e atribuição (var), entrada e saída (io), expressões e operadores aritméticos (exp), instruções condicionais (sel), laços e iteração (loops), coleções de dados (dc) e funções.

e métodos (func) e classes e objetos (objs) [49]. O instrumento e as perguntas originais foram criados em inglês e depois traduzidos para finlandês por um instrutor de idiomas especialista e um instrutor de programação especialista.

O volume complementar do CEFR para novos descritores [34] descreve três fases de validação para novas escalas de instrumentos: pesquisa e desenvolvimento iniciais (fase intuitiva), aprimoramento das categorias e da qualidade dos descritores (fase qualitativa) e calibração dos melhores descritores em uma escala matemática, além da confirmação dos pontos de corte entre os níveis (fase quantitativa). Nossa fase intuitiva consistiu em um trabalho exploratório para definir o número de níveis, subníveis e a formulação das afirmações pelos primeiro e segundo autores. A iteração da fase qualitativa foi conduzida pelos autores até que um ponto de saturação fosse atingido e um consenso geral fosse alcançado.

Nosso trabalho anterior [13] explorou as métricas de consistência interna em um piloto, analisando as respostas do protótipo SEI de 2196 alunos no início da semana 1. Esses resultados indicaram que o instrumento é altamente confiável (γ de Cronbach = 0,98) e pode se distinguir das métricas tradicionais [15], embora apresente forte correlação com elas.

Neste artigo, exploramos mais detalhadamente os dados quantitativos.

4. CONTEXTO DO ESTUDO E COLETA DE DADOS

Este estudo foi realizado durante um curso online de programação (disponível em <https://ohjelmointi-19.mooc.fi>), oferecido como um MOOC pela Universidade de Helsinque. O curso básico tem duração de sete semanas (dezembro a fevereiro) e abrange os princípios da programação em Java. Cada semana aborda os seguintes conceitos:

Semana 1: Entrada/Saída padrão, variáveis, condicionais, laços while.

Semana 2: Operadores lógicos, métodos e parâmetros, pilha de chamadas.

Semana 3: Listas e matrizes, loops, strings, testes.

Semana 4: Arquivos, sistema de arquivos, classes e objetos.

Semana 5: Variáveis primitivas e referenciais, sobrecarga de métodos, comparação de objetos, listas como objetos.

Semana 6: Classes e objetos, métodos, mapas de hash.

Semana 7: Classificação e pesquisa, testes unitários.

O curso é frequentado por alunos de graduação e não-acadêmicos da Universidade de Helsinque. Ao iniciar o curso, os alunos preenchem um formulário de consentimento para pesquisa e um questionário de informações pessoais (Duran et al. [13] exploram a relação entre o SEI e as perguntas sobre informações pessoais com mais detalhes). Responder ao questionário era voluntário, mas incluía um sorteio de ingressos de cinema como incentivo. Os alunos podiam pular qualquer pergunta e optar por não dar o consentimento para a pesquisa.

A idade média dos respondentes (n=3801) foi de 35,26 anos, dos quais 54,35% se identificaram como homens, 44,41% como mulheres, 0,63% não especificaram o sexo e 1,6% não responderam. A experiência prévia dos alunos em Ciência da Computação foi heterogênea (mediana = 10, média = 370,17, desvio padrão = 1897,87 horas totais programadas), assim como a formação acadêmica prévia em Ciência da Computação (mediana = 0, média = 1,68, desvio padrão = 4,23 cursos concluídos).

O SEI era uma tarefa opcional. Os dados foram coletados por meio de um formulário online que apresentava todos os níveis simultaneamente, e os alunos podiam pular qualquer conceito. Como é típico em MOOCs, a taxa de desistência foi alta e o número de participantes diminuiu ao longo das semanas.

O SEI foi aplicado no início da semana 1 (n=3807) e da semana 4 (n=1379) do curso, e no final da semana 7 (n=867).

O SCS1 (27 questões) foi traduzido para finlandês por um instrutor de idiomas especializado e um instrutor de programação especializado.

Em seguida, foi apresentada online como uma tarefa opcional no final da 7ª semana (simultaneamente com o SEI). Dos 640 alunos que responderam ao SCS1, 477 deram permissão para a realização de pesquisas com seus dados. Consideramos como participantes apenas aqueles que responderam a pelo menos 10 perguntas [58], resultando num total de 440 respondentes. No final da semana 7, os alunos (n=740) fizeram um exame online (3 questões, com pesos de 20%, 30% e 50%, respetivamente) para determinar a nota final do curso. Embora representasse apenas uma parte da nota final, a participação no exame era voluntária. O exame do curso consistia em três questões:

Q1 foi uma versão modificada do problema de precipitação [43] realizado com comprimentos de string.

A segunda questão pedia aos alunos que criassem um conjunto de métodos estáticos para criar e manipular listas e mapas.

Na questão 3, os alunos foram solicitados a criar um objeto (por exemplo, um músico) e, em seguida, outro objeto que contivesse tais objetos (por exemplo, uma banda). Além disso, esperava-se que os alunos construíssem um conjunto de métodos, desde métodos de obtenção de acesso até métodos que iterassem sobre os objetos aninhados, como, por exemplo, pedir aos músicos da banda que tocassem e apresentar a experiência média dos músicos.

Devido à natureza voluntária das tarefas, houve muita variação na forma como os alunos as realizaram. Um total de 283 alunos completou o SCS1 e a prova do curso, enquanto 261 também completaram o SEI na semana 7. Considerando todo o curso, 205 alunos completaram todas as tarefas na semana 7 e também responderam ao questionário de informações gerais e ao SEI na semana 1.

5 RESULTADOS

RQ1: Respostas dos alunos ao SEI e ao SCS1 Para investigar as

respostas dos alunos ao protótipo do SEI e do SCS1 (**RQ1**), comparamos as taxas de conclusão do SEI na Semana 7 com as do SCS1 e da prova do curso. Dos 867 respondentes que tentaram o SEI na Semana 7:

- 867 (100%) concluíram o SEI.
- 498 (57,43%) concluíram o exame.
- 376 (43,36%) responderam a mais de 10 questões do SCS1 (candidatos ao teste).
- 319 (36,79%) responderam a pelo menos metade das questões do SCS1.
- 84 (9,69%) responderam a todas as questões do SCS1.

Dos 740 participantes que tentaram fazer a prova do curso: • 740 (100%)

- concluíram a prova do curso.
- 498 (67,3%) concluíram o SEI.
- 254 (34,32%) responderam a mais de 10 questões do SCS1 (participantes do teste).
- 216 (29,19%) responderam a pelo menos metade das questões do SCS1.
- 55 (7,43%) responderam a todas as questões do SCS1.

Dos 477 respondentes que tentaram responder a pelo menos uma pergunta de SCS1:

- 422 (88,47%) concluíram o SEI.
- 283 (59,33%) concluíram o exame.
- 440 (99,24%) responderam a mais de 10 questões do SCS1 (candidatos ao teste).
- 365 (76,52%) responderam a pelo menos metade das questões do SCS1.
- 92 (19,29%) responderam a todas as questões do SCS1.

Os alunos responderam tanto ao SEI quanto ao SCS1 no mesmo sistema de pesquisa, e os links para os instrumentos foram inseridos lado a lado no material. Embora esperássemos que o questionário mais curto recebesse mais respostas do que o SCS1, mais extenso, ainda assim obtivemos resultados positivos.

Fiquei surpreso com a grande diferença no número de respondentes.

Para melhor compreender esse fenômeno, analisamos o feedback escrito recebido pela equipe do curso. Não houve nenhum feedback dos alunos sobre o SEI. Isso contrastou fortemente com o SCS1, sobre o qual os alunos se voluntariaram para fornecer uma quantidade considerável de feedback negativo. Por exemplo, alguns alunos reclamaram da falta de clareza no propósito da prova; uma discrepância entre as questões do SCS1 e o conteúdo do curso os levou a fazer suposições sobre os conceitos da linguagem. Alguns argumentaram que o SCS1 talvez estivesse testando a capacidade de ler o código de outras pessoas, algo que não é praticado no curso.

Os alunos também reclamaram do prazo de 1 hora para concluir a prova, argumentando que ela exige raciocínio muito rápido e boas habilidades de resolução de problemas.

Os alunos também afirmaram que os exercícios de preenchimento de lacunas eram bons e testavam sua compreensão lógica, mas as questões de rastreamento pareciam "testes de velocidade de processamento e memória". Questões que verificavam a saída de variáveis após chamadas de método foram consideradas difíceis, pois os alunos sentiram que esse comportamento é específico de uma determinada linguagem de programação e difícil de avaliar em pseudocódigo. Alguns até mencionaram que, se a dificuldade da prova do curso fosse tão alta quanto a do SCS1, poderiam até considerar desistir da disciplina. Em alguns casos, os alunos retiraram seu consentimento para pesquisa depois de verem o SCS1.

RQ2: Aumento da Confiança dos Alunos Para investigar

se a confiança dos alunos em sua capacidade de programação aumentou durante o curso (**RQ 2**), examinamos as estatísticas descritivas do SEI nas semanas 1, 4 e 7 (Tabela 1). A Tabela 2 mostra que as diferenças entre as médias de cada semana por conceito ($\bar{y}$) são todas significativas em $p < 0,001$, utilizando o teste U de Mann-Whitney. O tamanho do efeito (d de Cohen [10]) das mudanças da semana 1 para a semana 4 e da semana 1 para a semana 7 é grande. As mudanças da semana 4 para a semana 7 foram muito menores, exceto para classes e objetos, que apresentaram um grande tamanho de efeito. A Figura 1 mostra que esses resultados correspondem à cronologia do conteúdo do curso, particularmente o pequeno tamanho do efeito de $\bar{y}$ em todos os conceitos da semana 4 para a semana 7, exceto para classes e objetos, introduzidos na semana 4, que ainda podem ser considerados como um "novo conteúdo" para os alunos.

Figura 1: Cronograma do conteúdo e níveis médios dos conceitos do SEI.

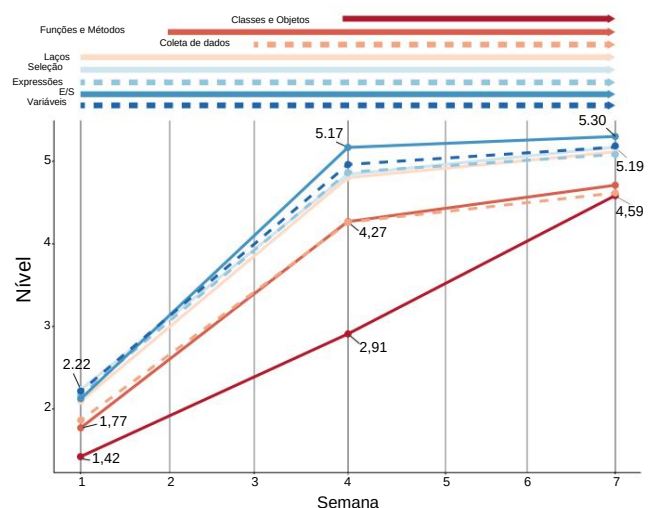


Tabela 1: Estatísticas descritivas dos conceitos do SEI.

	Semana 1										Semana										Semana 7			
	4 Variáveis E/S Experimentais Sel Loops DC Funct Obj Variáveis E/S Experimentais Sel Loops DC Funct Obj Variáveis E/S Experimentais Sel Loops DC Funct Obj										4 Variáveis E/S Experimentais Sel Loops DC Funct Obj Variáveis E/S Experimentais Sel Loops DC Funct Obj Variáveis E/S Experimentais Sel Loops DC Funct Obj										4 Variáveis E/S Experimentais Sel Loops DC Funct Obj Variáveis E/S Experimentais Sel Loops DC Funct Obj			
Média	2,22	2,13	2,17	2,23	2,10	2,91	5,19	5,30	5,08	5,07	5,11	4,59	1,42	4,96	5,17	4,86	4,84	4,80	4,26	4,27	4,62	4,71		
Desvio Padrão	2,06	2,04	2,02	2,0		1,98	1,85	1,84		1,72	1,14	0,96	1,13	1,09	1,10	1,21	1,32	1,98	1,03	0,91	1,08	0,96	0,99	
Mediana	2	2	2	2	2	2	1	1	1	1	5555	544	3	56555	55	5	5	5	5	5	5	5		
Instrumento Média						1,99										4,51					4,97			

Tabela 2: η^2 das médias por conceito SEI e tamanho do efeito de η^2 .

	Semana 4 - Semana 1	Semana 7 - Semana 4	Semana 7 - Semana 1
η^2 d de Cohen	η^2 d de Cohen	η^2 d de Cohen	η^2 d de Cohen
Nosso.	2,75* 2,97*	1,47† 1,55†	.23* .21†
Exp.	3,04*	1,67† 1,69†	.13* .14
E/S 2,7* Sel.		1,48† 1,55†	.22* .2
2,61* Loops		1,45† 1,59†	.33* .32†
2,71* 2,4*		1,51† 1,64†	.31* .29†
DC		1,41† 1,58†	.35* .3†
Func. 2.5*		1,46† 1,7†	.44* .35†
Obj. 1.49* =		0,83† 3,16†	.13† .98†

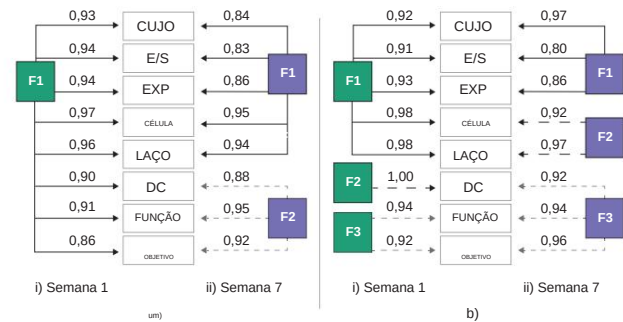
p < 0,001. † = efeitos grandes, ‡ = efeitos pequenos e = efeitos negligenciáveis.

RQ3: Diferenças de habilidade específicas para cada conceito

Investigar se o instrumento consegue distinguir entre diferenças específicas de conceito nas avaliações que os alunos fazem de sua capacidade (RQ3), Examinamos se as avaliações poderiam ser melhor compreendidas como únicas, ou modelos de múltiplos fatores. Realizamos uma Análise Fatorial Exploratória. Análise Fatorial Exploratória (AFE) para examinar possíveis estruturas fatoriais que se ajustem ao Dados SEI. Uma análise paralela utilizando o método de resíduos mínimos. Sugeriu-se entre 1 e 4 fatores para a semana 1 e entre 2 e 4 para a semana 4. fatores para as semanas 4 e 7.

Realizamos uma Análise Fatorial (AF) para investigar a estrutura dos conceitos. Acreditamos que os fatores estavam correlacionados e os dados A distribuição não era normal; utilizamos uma rotação oblíqua e o método de fatoração por Mínimos Quadrados Ordinários/Minres. Análise da Semana 1. determinou-se que apenas o modelo de um fator apresentou resultados aceitáveis. cargas fatoriais (>0,3), conforme mostrado na Figura 2(a)(i). Análise da semana 4 apresentaram cargas fatoriais aceitáveis usando 2, 3 e 4 fatores e semana O estudo 7 apresentou cargas fatoriais aceitáveis utilizando 2 e 3 fatores.

Figura 2: Cargas fatoriais nas semanas 1 e 7 para os dados gerais. (a) e estudantes experientes (n=51) (b).



Realizamos uma Análise Fatorial Confirmatória (AFC) ajustando o dados nos modelos FA sugeridos usando Máxima Verossimilhança (MV) como um método de estimação. Os parâmetros do modelo sugerem que o modelo de um fator se ajusta bem aos dados da semana 1 [20]. O Índice de Ajuste Comparativo

O valor de CFI de 0,92 mostra que o modelo de um fator apresenta um ajuste melhor do que um modelo de dois fatores. Modelo de base mais restritivo (valores > 0,9 são geralmente aceitos).

O Erro Quadrático Médio de Aproximação (RMSEA), que mede o quão bem o modelo reproduz os padrões dos dados (penalizando modelos mais complexos), tendo um valor de 0,11, que é muito próximo. ao valor sugerido de aceitação (η^2 0,1). Uma raiz padronizada O resíduo quadrático médio (SRMR) é uma medida absoluta de ajuste, definida como a diferença padronizada entre o observado e o previsto correlações. O modelo de um fator apresenta um bom ajuste aos dados. com um SRMR de 0,021 (η^2 0,08 é considerado um bom ajuste).

Como os modelos sugeridos para as semanas 4 e 7 apresentavam números e estruturas fatoriais diferentes, não puderam ser considerados. aninhados. Para testar qual modelo descreve os dados com maior precisão, Realizamos um teste de Vuong [53] e analisamos os estimadores de ajuste. quando necessário. O teste mostrou que o modelo de quatro fatores era distinguível dos outros modelos com menos fatores e foi considerado um ajuste melhor para os dados da semana 4 ($z = \eta^2$ 0,01, $p = 0,02$), e Todos os indicadores de ajuste do modelo podem ser considerados aceitáveis: CFI = 0,99 (>0,9), RMSEA = 0,063 (η^2 0,1), SRMR = 0,016 (η^2 0,08). Conceitos do fator um (cargas fatoriais) são: variáveis (0,85), entrada/saída (0,8) e expressões (0,88). Fator Dois conceitos são seleção (0,95) e loops (0,96). O terceiro fator consiste em de coleções de dados (1), enquanto funções (.93), e classes e objetos (.68) carregado no fator quatro.

Os testes de Vuong mostraram ainda que os modelos de três e dois fatores para A semana 7 não pôde ser considerada distinguível ($w^2 = 0,001$, $p = 0,54$), e os indicadores de parâmetros foram muito semelhantes para ambos os modelos. Em geral, modelos com Critério de Informação de Akaike (AIC) menor. e o Critério de Informação Bayesiano (BIC) pode ser considerado melhor ajuste aos dados. O modelo de três fatores apresentou valores menores (não significativos). valores para AIC (-1,46) e BIC (-6,23). No geral, ambos os modelos podem ser Aceito. Já que a única diferença entre as estruturas do modelo é um fator com um único componente (variáveis) com carga muito alta (1) No modelo de três fatores, sugerimos um modelo de dois fatores mais simples. ajuste, conforme apresentado na Figura 2(a)(ii).

Para examinar mais a fundo como o SEI poderia revelar conceitos específicos diferenças, investigamos se os fatores descritos anteriormente e As cargas permaneceriam mesmo ao focar em alunos especialistas (medidas) (com base no total de horas programadas relacionadas pelos próprios alunos, ou seja, HPT).

O teste de provocação oral (HPT) foi coletado na semana 1 e, nesta análise, consideramos apenas os casos em que o HPT foi coletado. alunos que concluíram todas as tarefas do curso ($n=205$).

Para alunos experientes (quartil superior), a análise fatorial exploratória (AFE) recomendou 1, 2 ou 3 fatores para a semana 1. O teste de Vuong mostrou que os três modelos são distinguíveis, o modelo de três fatores apresentou melhor ajuste ($z = \eta^2$ 0,98, $p = 0,16$), e seus indicadores estavam dentro os parâmetros recomendados (SRMR = 0,03, RMSEA = 0,16, CFI = .96, T LI = .94, $\eta^2 = 40,78$, $p = 0$). Para a Semana 7, a análise EFA sugeriram 2 ou 3 fatores. O teste de Vuong mostrou que os dois e Os modelos de três fatores não são distinguíveis, mas os indicadores do modelo de três fatores (SRMR = 0,03, RMSEA = 0,08, CFI = 0,99, T LI =

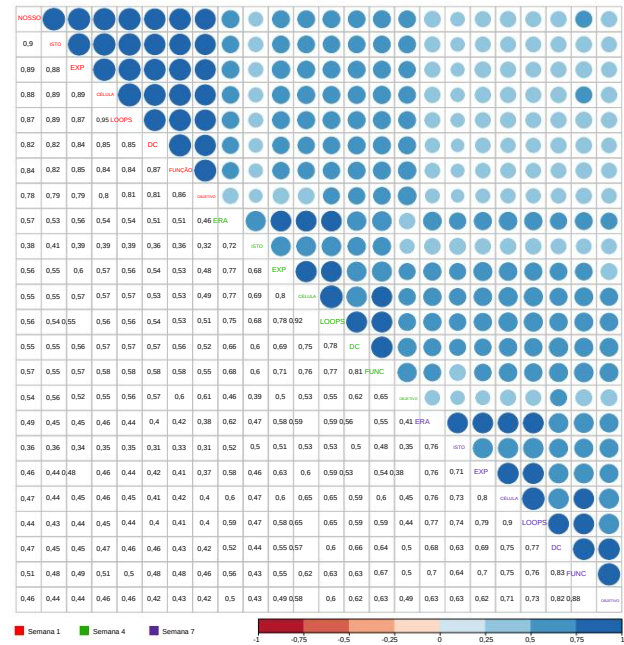
modelo $\chi^2 = 516,26$, $p = 0$, AIC = 374,57, BIC = 426,72) apresentaram um melhor ajuste de 0,98, χ^2 do que o de dois fatores (SRMR = 0,04, RMSEA = 0,15, CFI = 0,95, TLI = 0,93, $\chi^2 = 516,26$, $p = 0$, AIC = 388,99, BIC = 437,29). A Figura 2 (b) mostra as cargas fatoriais para os modelos de melhor ajuste para alunos experientes nas semanas 1 e 7.

RQ 4: Validade Externa Muitos

estudos investigaram o desempenho dos alunos usando o FCS1/SCS1 [23, 32, 50, 51, 58]. Uma das principais vantagens de usar um instrumento validado é produzir resultados comparáveis e generalizáveis, fornecendo informações significativas sobre o contexto investigado. Para estudar a validade externa do SEI (RQ4), primeiro examinamos a confiabilidade das métricas ao longo das semanas e, posteriormente, mostramos como o instrumento se correlaciona com a prova final do curso e com o SCS1.

As métricas de consistência interna e validade de construto demonstram que o SEI é muito confiável. O teste de confiabilidade alfa de Cronbach apresenta valores muito altos: $\bar{\gamma} = 0,98$ na semana 1, $\bar{\gamma} = 0,92$ na semana 4 e $\bar{\gamma} = 0,96$ na semana 7. A remoção de itens do instrumento não aumenta sua confiabilidade, exceto por um aumento de 0,03 ao remover classes e objetos na semana 4. A confiabilidade composta do instrumento também é muito alta: 0,98 para a semana 1, 0,94 para a semana 4 e 0,95 para a semana 7. A Figura 3 mostra as correlações de Spearman (corrigidas pelo método de Bonferroni para observações múltiplas) entre os conceitos ao longo do curso.

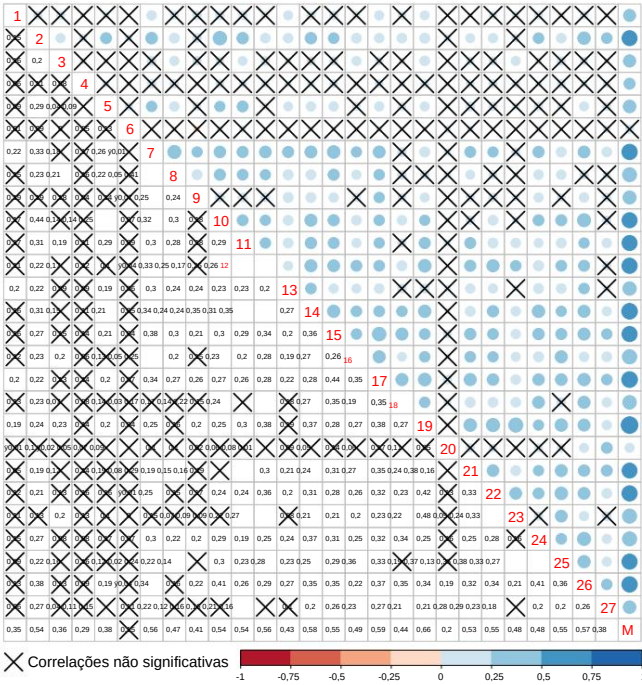
Figura 3: Correlações dos conceitos do SEI nas semanas 1, 4 e 7. O tamanho e a cor dos círculos correspondem ao grau de correlação.



A pontuação final padronizada (mín. = 0, máx. = 100) dos 440 participantes do teste SCS1 apresentou uma média de 36,98 (erro padrão de 1,06) pontos e um desvio padrão $\bar{\gamma} = 22,15$, semelhante aos resultados de Parker et al. (m = 35,85, $\bar{\gamma} = 13,1$) e superior ao de Timmermann et al. [50] (m = 22). O $\bar{\gamma}$ de Cronbach dos nossos dados do SCS1 ($\bar{\gamma} = 0,87$) foi superior aos valores de Xie et al. [58] ($\bar{\gamma} = 0,7$) e de Parker et al. [35] ($\bar{\gamma} = 0,59$). Ao examinar as mudanças de $\bar{\gamma}$ ao remover itens, a questão 6 foi a única considerada problemática com um aumento de 0,01 em $\bar{\gamma}$ (Xie et al.

[58] constataram que os itens 20, 24 e 27 eram problemáticos). Ao analisar as cargas fatoriais, verificamos que as questões 6 (0,08) e 20 (0,18) apresentaram cargas baixas, enquanto Xie et al. [58] relataram cargas baixas para as questões 20, 24 e 27. A Figura 4 mostra as correlações de Spearman (corrigidas por Bonferroni) de todos os 27 itens do SCS1 e sua pontuação média.

Figura 4: Correlações entre as questões do SCS1 (1-27) e a média do teste (M). Várias questões apresentaram correlações não significativas e podem ser consideradas problemáticas (ex.: 6, 20).



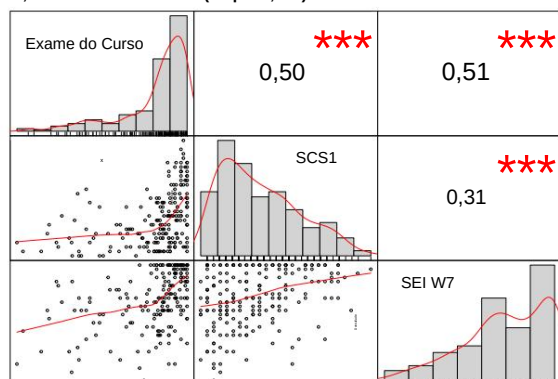
Os participantes da prova final do curso (n=740) obtiveram uma média de 76,16 pontos (erro padrão de 0,86) (mínimo = 3,33, máximo = 100) e um desvio padrão $\bar{\gamma} = 23,26$. O teste de confiabilidade alfa de Cronbach demonstrou que a prova do curso foi confiável ($\bar{\gamma} = 0,79$). Um teste de correlação de Spearman foi realizado para verificar a correlação entre cada questão de programação e a nota final, bem como entre as questões. A questão 1 apresentou correlação moderada com as questões 2 (0,46) e 3 (0,47), e a questão 2 apresentou forte correlação (0,62) com a questão 3 (todas com $p < 0,001$). As questões 1 (0,54), 2 (0,79) e 3 (0,82) apresentaram forte correlação com a nota da prova do curso ($p < 0,001$).

A Figura 5 apresenta a correlação de Spearman entre a prova do curso, SCS1 e SEI na semana 7 (n=205). O SEI na semana 7 e o SCS1 apresentaram uma correlação muito semelhante com as notas da prova do curso, enquanto que entre eles houve uma correlação moderada (0,31). Enquanto Parker et al. [35] não fornecem uma correlação do SCS1 com as notas do curso, Tew e Guzdial [49] relataram a mesma correlação (0,51) entre o FCS1 e a prova final.

RQ5: Replicação dos Resultados do SCS1

Como mencionado anteriormente, encontramos desempenho semelhante dos alunos no SCS1 e métricas semelhantes de confiabilidade interna do SCS1 em comparação com estudos anteriores. Agora, buscamos fornecer uma análise mais detalhada para investigar se nossos dados sugerem dificuldades nas mesmas questões (RQ5) por

Figura 5: Distribuição e correlações de Spearman entre o SCS1, exame do curso e SEI (*)** ($p < 0,001$)



Realizando uma análise da Teoria de Resposta ao Item (TRI) utilizando dados anteriores. pesquisa [35, 58] como parâmetros de referência para métodos e resultados.

Como o teste foi disponibilizado online, uma questão por vez, argumentamos que a independência condicional dos itens ainda é aceita [58]. Para testar a unidimensionalidade dos dados, um requisito da TRI (Teoria de Resposta ao Item), nós Realizou-se uma Análise Fatorial Confirmatória (AFC), ajustando os dados. em um modelo de fator latente único usando mínimos quadrados ponderados (WLSMV) como método de estimação [58]. O modelo latente único Os indicadores (CFI = 0,931, RMSEA = 0,039 e SRMR = 0,048) sugerem um ajuste aceitável, muito próximo dos valores de Xie et al. [58] (CFI = 0,98, RMSEA = 0,014 e SRMR = 0,079).

Um teste ANOVA confirmou que um modelo de três parâmetros (3PL) era uma opção melhor do que os modelos com restrições, com todos os itens se encaixando perfeitamente em nossos dados. O SCS1 pode ser considerado moderadamente difícil, com uma média de dificuldade $\bar{y}_j = 0,81$ (Tabela 3). A habilidade dos alunos ($\bar{y}$) variou de $\bar{y} = [1,74, 2,76]$ com um valor médio de 0,018. O primeiro quartil dos alunos apresentou uma estimativa de habilidade $\bar{y} = \bar{y}_{0,65}$, enquanto o quarto dos alunos apresentou uma estimativa de habilidade $\bar{y} = 0,66$. Todos os valores de discriminação ($\bar{y}_j$) podem ser considerados significativos ($>0,4$) [58]. Nossa análise utilizou um 3PL (em oposição ao 2PL de Xie et al.). As perguntas apresentaram um valor positivo no terceiro parâmetro, que pode ser interpretado como um valor de pseudo-avinhiação onde os alunos em níveis mais baixos de habilidade não apresentam mais probabilidade zero de ter uma resposta correta, mas em vez de uma probabilidade (y_j).

A Tabela 3 mostra que as questões 6 e 20 foram as mais difíceis ($\bar{y}$)), mas nenhuma questão foi considerada muito difícil ($\bar{y} > 3$), ao contrário do que ocorre com as questões anteriores. aos resultados de Xie et al. [58] ($\bar{y}_k$), onde as questões 5, 13, 15 e 18 foram considerado muito difícil. Usando a metodologia de Parker et al. [35] (% de respostas corretas), questões 3 (0,8), 11 (0,51), 12 (0,55), 19 (0,61) e 23 (0,64) poderia ser considerado moderadamente difícil e todos os outros itens difícil. Parker et al. descobriram que as questões 1, 2, 3, 19 e 23 poderiam ser Considerada moderadamente difícil e todas as outras questões difíceis.

6 DISCUSSÃO

Nossa RQ1 dizia respeito às respostas dos alunos ao SEI e ao SCS1. Nós esperava-se que os alunos tivessem maior probabilidade de se voluntariar para responder às perguntas. SEI do que fazer um teste, o que era claramente o caso. Mesmo quando comparado ao exame do curso (que era opcional, mas afetava a nota),

O número de respostas SEI é quase duas vezes maior do que para o

Tabela 3: Parâmetros das questões SCS1 usando um modelo 3PL.

[illegible]

SCS1. Possíveis explicações para tal comportamento incluem o fato de que os alunos geralmente não gostam de ser testados, mesmo quando não há notas [19]; uma incompatibilidade entre o conteúdo do curso e o que o SCS1 mede; baixa sensibilidade do pseudocódigo; ou mesmo a perspectiva de ter um teste. A baixa pontuação no teste será armazenada para pesquisas futuras. Sugerimos que o SEI seja utilizado. Pode reduzir a ansiedade em relação aos testes, proporcionando mais oportunidades para os alunos. Isso pode permitir que os alunos reflitam sobre si mesmos e aprimorem suas habilidades metacognitivas [38] também conforme planejam seu aprendizado.

Embora não tenham coletado informações específicas sobre o

Aplicação SEI (ex.: feedback e tempo gasto na tarefa), voluntários dos alunos

Fornecer feedback (negativo) sobre o SCS1. Uma fraqueza óbvia.

Uma desvantagem dos testes é que, embora potencialmente mais precisos, podem ser uma tarefa demorada e exaustiva (apenas 19,29% dos que tentaram).

O SCS1 completou-o na íntegra). Os alunos comentaram particularmente sobre o curto limite de tempo do SCS1 (opinião compartilhada pelos autores).

Pesquisas recentes mostram que um esforço mental intenso pode interferir com a aprendizagem através do esgotamento de recursos cognitivos [8]. Nossos resultados sugerem que o SCS1 pode desmotivar os alunos; portanto, incentivamos abordagens que aliviam esse comportamento, como autoavaliações ou versões mais leves do SCS1 [6]. Dada a natureza simplificada do SEI, com base nas taxas de conclusão extremamente altas (ou seja, todos) (quem tentou o SEI também o concluiu) e correlações comparáveis com as notas do curso, sugerimos que o SEI pode ser uma ferramenta eficiente

Ferramenta adicional no processo de avaliação da aprendizagem.

A crescente confiança dos alunos, medida pelo SEI, corresponde às expectativas. a estrutura do curso (por exemplo, tamanhos de efeito muito limitados da semana 4 à 7) em todos os conceitos, exceto nos ainda "novos". (Ver Figura 1). Futuro

A pesquisa deve investigar se esse aumento e discriminação

A relação entre os conceitos seria ainda mais evidente em um curso.

Cronograma que introduz conceitos em um ritmo mais lento. Os resultados sugerem que o instrumento pode distinguir entre conceitos e que essa distinção melhora com medições repetidas, o que corrobora as descobertas de que a autoavaliação pode melhorar com prática [38]. Embora a semana 1 tenha apresentado cargas semelhantes para todos os conceitos (exceto para classes e objetos. Veja a Figura 2), a distinção entre

Os conceitos tornaram-se mais pronunciados e a estrutura fatorial

isso mudou à medida que os alunos se tornaram mais experientes e adquiriram mais conhecimento.

com o conteúdo. Cargas fatoriais e estrutura da experiência

Os alunos na primeira semana são semelhantes a toda a turma no final da semana.

curso. Isso implica que o SEI discrimina conceitos precocemente e não apenas seguir o cronograma do conteúdo do curso.

As métricas de consistência interna mostraram que as medidas repetidas do SEI não diminuem sua confiabilidade e permanecem superiores às dos testes (Figura 3). Em termos de desempenho, os resultados do SCS1 mostram que nosso contexto é comparável a pesquisas anteriores (Tabela 3). No entanto, sabemos pouco sobre a capacidade de autoavaliação desses alunos. Assim, é difícil determinar sua precisão e compará-los com outras coortes, e podemos oferecer apenas afirmações limitadas de transferibilidade.

Embora a correlação do SEI com a prova tenha sido marginalmente forte, o mesmo ocorreu com o SCS1 (Figura 5). Ross [38] sugere que tais comparações devem ser feitas com cautela, uma vez que as provas de cursos podem ser tendenciosas. Professores e alunos podem ter expectativas diferentes sobre a adequação de uma prova para avaliar a aprendizagem do conteúdo do curso. Em nosso caso, ressaltamos que, embora tanto o SCS1 quanto o protótipo do SEI se concentrem em habilidades de compreensão de código (ou conceito), a prova final do curso foi um teste de escrita de programa.

Devido às baixas correlações internas e ao feedback dos alunos (ver Figura 4), os resultados do SCS1 podem ser difíceis de interpretar, e muitas questões com vários conceitos inter-relacionados dificultam a avaliação do conhecimento dos alunos. Outras abordagens apresentaram correlações mais altas com as provas do curso (por exemplo, [1]), mas exigem um conhecimento extenso e dispendioso dos dados dos alunos. Alguns alunos também argumentaram que o SCS1 apresenta baixa adequação ao conteúdo do curso. Além disso, testes validados são menos flexíveis e difíceis de adaptar a outros contextos, enquanto o instrumento SEI foi projetado para ser flexível e permitir personalização. Discutimos ainda possibilidades para aprimorar o SEI em nossas conclusões.

Embora o desempenho do nosso grupo no SCS1 esteja de acordo com pesquisas anteriores, o mesmo não foi observado na análise das respostas questão por questão. A análise IRT mostrou que não apenas a dificuldade das questões foi classificada de forma diferente, como também algumas questões apresentaram níveis de dificuldade e desempenho bastante distintos (Tabela 3). Por exemplo, a questão 3 pode ser considerada muito fácil para o nosso grupo, mas moderadamente difícil para o grupo de Xie et al. [58]. Por outro lado, observamos o fenômeno oposto na questão 6. A questão 18 foi difícil para o nosso grupo, mas muito difícil para os alunos de Xie et al. (o mesmo se aplica às questões 13 e 15). Embora uma análise de desempenho e dificuldade possa revelar diferenças entre os grupos, essa análise fica incompleta sem a discussão das próprias questões da prova. O que torna a questão 6 tão difícil para um grupo e não para outro? Seria um caso de estrutura e conteúdo do curso diferentes, ou talvez um fenômeno distinto. Por essa razão, sugerimos que, no futuro, a comunidade de Pesquisa Comparativa em Relação à Eficácia (CER) não apenas examine o desempenho nos testes, mas também analise as questões dos testes para compreender esse fenômeno.

7 CONCLUSÃO

Ao projetar o SEI, nosso objetivo era criar uma ferramenta leve que fornecesse informações relevantes para alunos e instrutores, funcionando também como uma ferramenta de autoavaliação para os alunos, apoiando a aprendizagem autorregulada. Nossos resultados atuais corroboram a ideia de que o SEI pode ser uma ferramenta útil e eficiente, mesmo como protótipo. Ele proporciona uma discriminação justa de conceitos, captura a crescente confiança dos alunos em sua capacidade de compreensão de código, é internamente confiável e coerente, permite medições repetidas sem perder sua validade e pode ser disponibilizado publicamente. Nossa estrutura preliminar demonstra potencial para o desenvolvimento de outros SEIs que possam ser facilmente escalados e aplicados. Além disso, permite que alunos de diversas origens

Para ser avaliada com relativa precisão, proporciona aos alunos a oportunidade de refletir sobre si mesmos sem o peso de se sentirem testados.

No entanto, o instrumento e nossos resultados apresentam algumas limitações. A validação do instrumento mostra que o protótipo do SEI atingiu parcialmente alguns desses objetivos. Medidas de validade divergente (por exemplo, comparando o SEI com instrumentos de autoeficácia) poderiam fornecer suporte adicional para um construto distinto. Certamente existe uma sobreposição entre autoeficácia e autoavaliação. Nossos resultados mostram que, assim como Danielsiek et al. [12], há um aumento na percepção dos alunos sobre a habilidade relacionada à programação e fatores distintivos no instrumento. Enquanto Danielsiek et al. [12] avaliam habilidades como projeto de algoritmos, análise de tempo de execução e escrita e rastreamento de pseudocódigo, nosso instrumento avalia a capacidade de compreender conceitos de programação.

O SEI ainda requer evidências mais robustas de validação externa.

Embora nossos resultados tenham sido comparáveis aos do SCS1 nesse aspecto, acreditamos que tanto o SEI quanto os testes poderiam ser aprimorados para fornecer informações mais confiáveis e precisas. A prova final utilizou questões de escrita de código, enquanto o SEI avalia a capacidade de compreensão de código, o que poderia explicar os níveis de correlação observados em nossos resultados. Pesquisas futuras também devem buscar comparar o SEI e o SCS1 com provas de cursos de compreensão de código.

Além da conceitualização e avaliação do SEI, fornecemos uma análise abrangente de uma replicação do SCS1. Essa análise corrobora as descobertas de pesquisas anteriores, oferece uma comparação detalhada entre contextos bastante distintos utilizados em trabalhos prévios e levanta questões importantes sobre os instrumentos de CER validados atualmente e como a comunidade interpreta seus resultados, oferecendo feedback construtivo aos desenvolvedores de instrumentos. Propomos que futuros estudos de replicação, utilizando ferramentas validadas como referência, examinem cuidadosamente o desempenho em nível de questão e as diferenças de desempenho, bem como investiguem o conteúdo das questões para apresentar argumentos que justifiquem tais diferenças.

Este estudo do protótipo SEI mostra seu potencial para delinear uma estrutura independente de contexto que os instrutores podem usar para criar SEIs comparáveis, adaptados a necessidades específicas. Seguindo as diretrizes de Ross [38] para melhorar a autoavaliação e os resultados de pesquisas anteriores [30], recomendamos trabalhos futuros para melhorar a confiabilidade e a validade de instrumentos direcionados a um contexto específico (linguagem e comportamento).

Instrumentos personalizados podem incluir a customização do número de níveis (por exemplo, o nível A0 pode ser significativo para alunos do Ensino Fundamental I (K-8), mas não para instrutores de Introdução à Ciência da Computação); o número de subníveis com base na complexidade esperada do código com o qual os alunos normalmente trabalham; e o fornecimento de exemplos contextualizados. O número e os tipos de conceitos no SEI também devem refletir o conteúdo e o contexto do curso; por exemplo, um SEI para um curso de Introdução à Ciência da Computação com foco em funcionalidade abordaria conceitos diferentes.

Analizamos apenas um único curso em um contexto de MOOC, o que limita as generalizações possíveis. Em validações futuras, pretendemos aprimorar a clareza dos descritores coletando feedback dos alunos, conduzindo investigações qualitativas sobre o raciocínio dos alunos ao responderem ao instrumento e replicando este trabalho em outros contextos.

8 AGRADECIMENTOS

Agradecemos aos autores do SCS1 [35] por o disponibilizarem para nosso uso e a Benjamin Xie pelas discussões sobre os métodos utilizados em sua análise IRT e resultados do SCS1 [58].

REFERÊNCIAS

[1] Alireza Ahadi, Raymond Lister, Heikki Haapala e Arto Vihavainen. 2015. Explorando métodos de aprendizado de máquina para identificar automaticamente alunos que precisam de assistência. In *Anais da Décima Primeira Conferência Internacional Anual sobre Pesquisa em Educação Computacional (ICER '15)*. ACM, Nova York, NY, EUA, 121–130. <https://doi.org/10.1145/2787622.2787717>

[2] Satu Alaoitinen e Kari Smolander. 2010. Autoavaliação do aluno em um curso de programação usando a taxonomia revisada de Bloom. Em *Anais da Décima Quinta Conferência Anual sobre Inovação e Tecnologia na Educação em Ciência da Computação (ITICSE '10)*. ACM, Nova York, NY, EUA, 155–159. <https://doi.org/10.1145/1822090.1822135>

[3] Lorin W Anderson, David R Krathwohl, Peter W. Airasian, Kathleen A. Cruikshank, Richard E. Mayer, Paul R. Pintrich, James Rath e Merlin C. Wittrock. 2001. Uma taxonomia para aprendizagem, ensino e avaliação: Uma revisão da taxonomia de objetivos educacionais de Bloom (edição abreviada). Addison Wesley Longman. 302 páginas.

[4] Albert Bandura. 1977. Autoeficácia: em direção a uma teoria unificadora da mudança comportamental. *Psychological Review* 84, 2 (1977), 191–215. [https://doi.org/10.1016/0146-6402\(78\)90002-4](https://doi.org/10.1016/0146-6402(78)90002-4) [5] Jens F. Beckmann. 2010. Domando uma besta de carga - Sobre algumas questões com a conceitualização e operacionalização da carga cognitiva. *Aprendizagem e Instrução* 20, 3 (2010), 250 – 264. <https://doi.org/10.1016/j.learninstruc.2009.02.024> [6] Ryan Bockmon, Stephen Cooper, Jonathan Gratch e Mohsen Dorodchi. 2019. (Re)Validação de instrumentos cognitivos introdutórios de computação. In *Anais do 50º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (SIGCSE '19)*. ACM, Nova Iorque, NY, EUA, 552–557. <https://doi.org/10.1145/3287324.3287372> [7] Janet Carter, Su White, Karen Fraser, Stanislav Kurkovsky, Colette McCreesh e Malcolm Wieck. 2010. Relatório do Grupo de Trabalho ITICSE 2010: Motivando Nossos Melhores Alunos. Em *Anais dos Relatórios do Grupo de Trabalho ITICSE 2010 (ITICSE-WGR '10)*. ACM, Nova Iorque, NY, EUA, 29–47. <https://doi.org/10.1145/1971681.1971685> [8] Ouhao Chen, Juan C. Castro-Alonso, Fred Paas e John Sweller. 2018. Extending Cognitive Load Theory to Incorporate Working Memory Resource Depletion: Evidence from the Spacing Effect. *Educational Psychology Review* 30, 2 (01 Jun 2018), 483–501. <https://doi.org/10.1007/s10648-017-9426-2> [9] Tony Clear, Anne Philpott, Phil Robbins e Simon. 2009. Report on the Eighth BRACElet Workshop: BRACElet Technical Report 01/08. *Bulletin of Applied Computing and Information Technology* 7, 1 (2009).

[10] Jacob Cohen. 2013. *Análise de poder estatístico para as ciências comportamentais*. Routledge.

[11] Conselho da Europa. 2001. Quadro Europeu Comum de Referência para as Línguas : Aprendizagem, ensino, avaliação. Estrasburgo. <https://rm.coe.int/1680459f97> [12] Holger Danielsiek, Laura Toma e Jan Vahrenhold. 2017. Um instrumento para avaliar a autoeficácia em cursos introdutórios de algoritmos. Em *Anais da Conferência ACM de 2017 sobre Pesquisa Internacional em Educação em Computação (ICER '17)*. ACM, Nova Iorque, NY, EUA, 217–225. <https://doi.org/10.1145/3105726.3106171> [13] Rodrigo Duran, Jan-Mikael Rybicki, Arto Hellas e Sanna Suoranta. 2019. Em direção a um instrumento comum para medir o conhecimento prévio de programação. Em *Anais da 24ª Conferência Anual da ACM sobre Inovação e Tecnologia no Ensino de Ciência da Computação (ITICSE 2019)*. ACM, Nova York, NY, EUA, 6. <https://doi.org/10.1145/3304221.3319755>

[14] Rodrigo Duran, Juha Sorva e Sofia Leite. 2018. Rumo a uma análise da complexidade de programas a partir de uma perspectiva cognitiva. In *Anais da Conferência ACM de 2018 sobre Pesquisa Internacional em Educação Computacional (ICER '18)*. ACM, Nova York, NY, EUA, 21–30. <https://doi.org/10.1145/3230977.3230986> [15] J. Feigenspan, C. Kästner, J. Liebig, S. Apel e S. Hanenberg. 2012. Medindo a experiência de programação. In 20ª Conferência Internacional IEEE sobre Compreensão de Programas (ICPC). 73–82. <https://doi.org/10.1109/ICPC.2012.6240511> [16] Ken Goldman, Paul Gross, Cinda Heeren, Geoffrey L. Herman, Lisa Kaczmarczyk, Michael C. Loui e Craig Zilles. 2010. Definindo o Escopo dos Inventários de Conceitos para Disciplinas Introdutórias de Computação. *Trans. Comput. Educ.* 10, 2, Artigo 5 (junho de 2010), 29 páginas. <https://doi.org/10.1145/1789934.1789935> [17] Shuchi Grover, Stephen Cooper e Roy Pea. 2014. Avaliando a Aprendizagem Computacional no Ensino Fundamental e Médio. Nos *Anais da Conferência de Inovação e Tecnologia na Educação em Ciência da Computação de 2014 (ITICSE '14)*. ACM, Nova Iorque, NY, EUA, 57–62. <https://doi.org/10.1145/2591708.2591713> [18] Dianne Hagan e Selby Markham. 2000. É útil ter alguma experiência em programação antes de iniciar um curso de graduação em computação? In *Anais da 5ª Conferência Anual SIGCSE/SIGCUE ITICSE sobre Inovação e Tecnologia no Ensino de Ciência da Computação (ITICSE '00)*. ACM, Nova Iorque, NY, EUA, 25–28. <https://doi.org/10.1145/343048.343063> [19] Ray Hembree. 1988. Correlações, causas, efeitos e tratamento da ansiedade em testes. Revisão de pesquisa educacional 58, 1 (1988), 47–77.

[20] Li-tze Hu e Peter M Bentler. 1999. Critérios de corte para índices de ajuste na análise de estrutura de covariância: critérios convencionais versus novas alternativas. *Modelagem de equações estruturais: um periódico multidisciplinar* 6, 1 (1999), 1–55.

[21] Maria Kallia e Sue Sentance. 2019. Aprendendo a usar funções: a relação entre concepções errôneas e autoeficácia. In *Anais do 50º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (SIGCSE '19)*. ACM, Nova York, NY, EUA, 752–758. <https://doi.org/10.1145/3287324.3287377>

[22] Danny Kostons, Tamara van Gog e Fred Paas. 2012. Treinamento de habilidades de autoavaliação e seleção de tarefas: uma abordagem cognitiva para melhorar a aprendizagem autorregulada . *Aprendizagem e Instrução* 22, 2 (2012), 121–132. <https://doi.org/10.1016/j.learninstruc.2011.08.004>

[23] Michael J. Lee e Andrew J. Ko. 2015. Comparando a eficácia de abordagens de aprendizagem online nos resultados de aprendizagem de CS1. Em *Anais da Décima Primeira Conferência Internacional Anual sobre Pesquisa em Educação em Computação (ICER '15)*. ACM, Nova York, NY, EUA, 237–246. <https://doi.org/10.1145/2787622.2787709>

[24] Elizabeth A. Linnenbrink e Paul R. Pintrich. 2003. O papel das crenças de autoeficácia no envolvimento e na aprendizagem dos alunos em sala de aula. *Reading & Writing Quarterly* 19, 2 (2003), 119–137. <https://doi.org/10.1080/10573560308223>

[25] Raymond Lister. 2016. Rumo a uma epistemologia desenvolvimentista da programação de computadores. In *Anais do 11º Workshop em Educação em Computação no Ensino Fundamental e Médio (WIPSCSE '16)*. ACM, Nova York, NY, EUA, 5–16. <https://doi.org/10.1145/2978249.2978251> [26] Mike Lopez, Jacqueline Whalley, Phil Robbins e Raymond Lister. 2008. Relações entre habilidades de leitura, rastreamento e escrita em programação introdutória. In *Anais do Quarto Workshop Internacional sobre Pesquisa em Educação Computacional (ICER '08)*. ACM, Nova York, NY, EUA, 101–112. <https://doi.org/10.1145/1404520.1404531> [27] Andrew Luxton-Reilly, Brett A. Becker, Yingjun Cao, Roger McDermott, Claudio Mirolo, Andreas Mühling, Andrew Petersen, Kate Sanders, Simon e Jacqueline Whalley. 2017. Desenvolvimento de avaliações para determinar o domínio dos fundamentos de programação. Em *Anais da Conferência ITICSE de 2017 sobre Relatórios de Grupos de Trabalho (ITICSE-WGR '17)*. ACM, Nova York, NY, EUA, 47–69. <https://doi.org/10.1145/3174781.3174784> [28] Lauren Margulieux, Tuba Ayer Ketenci e Adrienne Decker. 2019. Revisão das métricas utilizadas em pesquisas sobre o ensino da computação e sugestões para aumentar a padronização. *Educação em Ciência da Computação* 29, 1 (2019), 49–78. <https://doi.org/10.1080/08993408.2018.1562145> <https://doi.org/10.1080/08993408.2018.1562145>

[29] Susana Masapanta-Carrión e J. Ángel Velázquez-Ilturbide. 2018. Uma revisão sistemática do uso da Taxonomia de Bloom no ensino de Ciência da Computação. In *Anais do 49º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (SIGCSE '18)*. ACM, Nova York, NY, EUA, 441–446. <https://doi.org/10.1145/3159450.3159491>

[30] Peter J. Miller. 2003. O efeito da especificidade dos critérios de pontuação na avaliação e avaliação Avaliação por pares e autoavaliação. no ensino superior - <https://doi.org/10.1080/0260293032000066218> <https://doi.org/10.1080/0260293032000066218> <https://doi.org/10.1080/0260293032000066218> [31] Laurie Murphy e Josh Tenenber. 2005. Os alunos de Ciência da Computação sabem o que sabem?: Um estudo de calibração do conhecimento de estrutura de dados. Em *Anais da 10ª Conferência Anual SIGCSE sobre Inovação e Tecnologia no Ensino de Ciência da Computação (ITICSE '05)*. ACM, Nova York, NY, EUA, 148–152. <https://doi.org/10.1145/1067445.1067488>

[32] Greg L. Nelson, Benjamin Xie e Andrew J. Ko. 2017. Compreensão em primeiro lugar: avaliando uma nova pedagogia e sistema de tutoria para rastreamento de programas em CS1. Em *Anais da Conferência Internacional de Pesquisa em Educação Computacional da ACM de 2017 (ICER '17)*. ACM, Nova York, NY, EUA, 2–11. <https://doi.org/10.1145/3105726.3106178>

[33] Grace Ngai, Winnie WY Lau, Stephen CF Chan e Hong-va Leong. 2010. Sobre a implementação da autoavaliação em um curso introdutório de programação. *SIGCSE Bull.* 41, 4 (jan. 2010), 85–89. <https://doi.org/10.1145/1709424.1709453> [34] Conselho da Europa. 2018 (acessado em 1 de abril de 2019). Quadro Europeu Comum de Referência para Línguas: Aprendizagem, Ensino, Avaliação. Volume complementar com novos descritores. <https://rm.coe.int/celf-companion-volume-with-new-descriptors-2018/1680787989> [35] Miranda C. Parker, Mark Guzdial e Shelly Engleman. 2016. Replicação, Validação e Uso de uma Avaliação de Conhecimento CS1 Independente de Idioma. Em *Anais da Conferência ACM de 2016 sobre Pesquisa Internacional em Educação em Computação (ICER '16)*. ACM, Nova York, NY, EUA, 93–101. <https://doi.org/10.1145/2960310.2960316>

[36] Leo Porter, Cynthia Taylor e Kevin C. Webb. 2014. Aproveitando os princípios do código aberto para o desenvolvimento flexível do inventário de conceitos. In *Anais da Conferência de Inovação e Tecnologia na Educação em Ciência da Computação de 2014 (ITICSE '14)*. ACM, Nova York, NY, EUA, 243–248. <https://doi.org/10.1145/2591708.2591722> [37] Marcos Román-González, Juan-Carlos Pérez-González e Carmen Jiménez-Fernández. 2017. Quais habilidades cognitivas estão subjacentes ao pensamento computacional? Validade de critério do Teste de Pensamento Computacional. *Computers in Human Behavior* 72 (2017), 678 – 691. <https://doi.org/10.1016/j.chb.2016.08.047> [38] John A. Ross. 2006. A confiabilidade, validade e utilidade da autoavaliação. *Avaliação prática, pesquisa e avaliação* 10 (2006), 1–13. <https://doi.org/10.1016/j.aspen.2014.06.014>

[39] Sam Saarinen, Shriram Krishnamurthi, Kathi Fisler e Preston Tunnell Wilson. 2019. Aproveitando a sabedoria das turmas: Classsourcing e aprendizado de máquina para geração de instrumentos de avaliação. In *Anais da 50ª Conferência Técnica da ACM*

- Simpósio sobre Educação em Ciência da Computação (SIGCSE '19). ACM, Nova York, NY, EUA, 606–612. <https://doi.org/10.1145/3287324.3287504>
- [40] Dale H Schunk. 1981. Modelagem e efeitos atribucionais no desempenho infantil : uma análise de autoeficácia. *Journal of educational psychology* 73, 1 (1981), 93. <https://doi.org/10.1037/0022-0663.73.1.93>
- [41] Michael James Scott e Gheorghita Ghinea. 2014. Medindo o Enriquecimento: A Montagem e Validação de um Instrumento para Avaliar as Autoconfianças dos Alunos em CS1. In *Anais da Décima Conferência Anual sobre Pesquisa Internacional em Educação Computacional (ICER '14)*. ACM, Nova York, NY, EUA, 123–130. <https://doi.org/10.1145/2632320.2632350>
- [42] Judy Sheard, Angela Carbone, Selby Markham, AJ Hurst, Des Casey e Chris Avram. 2008. Desempenho e Progressão de Alunos do Primeiro Ano de TIC. Em *Anais da Décima Conferência sobre Educação em Computação da Australásia - Volume 78 (ACE '08)*. Australian Computer Society, Inc., Darlinghurst, Austrália, 119–127. <http://dl.acm.org/citation.cfm?id=1379249.1379261>
- [43] E. Soloway. 1986. Aprender a Programar = Aprender a Construir
- Mecanismos e Explicações. *Commun. ACM* 29, 9 (set. 1986), 850–858. <https://doi.org/10.1145/6592.6594>
- [44] John Sweller. 2010. Interatividade de elementos e carga cognitiva intrínseca, extrínseca e pertinente .
- Educational Psychology Review* 22, 2 (01 de junho de 2010), 123–138. <https://doi.org/10.1007/s10648-010-9128-5>
- [45] C. Taylor, D. Zingaro, L. Porter, KC Webb, CB Lee e M. Clancy. 2014. Inventários de conceitos de ciência da computação: passado e futuro. *Computer Science Education* 24, 4 (2014), 253–276. <https://doi.org/10.1080/08993408.2014.970779> arXiv:<https://doi.org/10.1080/08993408.2014.970779> [46] Harriet G. Taylor e Luegina
- C. Mounfield. 1994. Exploração da relação entre experiência prévia em computação e gênero no sucesso em ciência da computação na faculdade. *Journal of Educational Computing Research* 11, 4 (1994), 291–306. <https://doi.org/10.2190/4U0A-36XP-EU5K-H4KV> arXiv:<https://doi.org/10.2190/4U0A-36XP-EU5K-H4KV>
- [47] Donna Teague. 2015. Teoria neo-piagetiana e o programador novato. Tese de doutorado. Dissertação. Universidade de Tecnologia de Queensland.
- [48] Donna Teague, Malcolm Corney, Alireza Ahadi e Raymond Lister. 2013. Um estudo qualitativo de pensamento em voz alta dos estágios iniciais do raciocínio neopiagetiano em programadores iniciantes. In *Anais da Décima Quinta Conferência Australiana de Educação em Computação - Volume 136 (ACE '13)*. Australian Computer Society, Inc., Darlinghurst, Austrália, 87–95. <http://dl.acm.org/citation.cfm?id=2667199.2667209>
- [49] Allison Elliott Tew e Mark Guzdial. 2010. Desenvolvimento de uma avaliação validada de conceitos fundamentais de CS1. In *Anais do 41º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (SIGCSE '10)*. ACM, Nova York, NY, EUA, 97–101. <https://doi.org/10.1145/1734263.1734297>
- [50] D. Timmermann, C. Kautz e V. Skwarek. 2016. Redesenho baseado em evidências de um curso introdutório de "programação em C". Em 2016 IEEE Frontiers in Education Conference (FIE). 1–5. <https://doi.org/10.1109/FIE.2016.7757492>
- [51] Ian Utting, Allison Elliott Tew, Mike McCracken, Lynda Thomas, Dennis Bouvier , Roger Frye, James Paterson, Michael Caspersen, Yifat Ben-David Kolikant, Juha Sorva e Tadeusz Wilusz. 2013. Uma nova perspectiva sobre o desempenho de programadores iniciantes e as expectativas de seus professores. In *Anais da Conferência de Relatórios do Grupo de Trabalho ITICSE sobre Inovação e Tecnologia no Ensino de Ciência da Computação (ITICSE-WGR '13)*. ACM, Nova York, NY, EUA, 15–32. <https://doi.org/10.1145/2543882.2543884> [52] Arto Vihavainen, Jonne Airaksinen e Christopher Watson. 2014. Uma revisão sistemática de abordagens para o ensino de programação introdutória e sua influência no sucesso. Em *Anais da Décima Conferência Anual sobre Pesquisa Internacional em Educação em Computação (ICER '14)*. ACM, Nova York, NY, EUA, 19–26. <https://doi.org/10.1145/2632320.2632349>
- [53] Quang H. Vuong. 1989. Testes de razão de verossimilhança para seleção de modelos e hipóteses não aninhadas. *Econometrica* 57, 2 (1989), 307–333. <https://doi.org/10.2307/1912557>
- [54] Christopher O. Walker, Barbara A. Greene e Robert A. Mansell. 2006. Identificação com acadêmicos, motivação intrínseca/extrínseca e autoeficácia como preditores de engajamento cognitivo. *Aprendizagem e Diferenças Individuais* 16, 1 (2006), 1 – 12. <https://doi.org/10.1016/j.lindif.2005.06.004>
- [55] David Weintrop e Uri Wilensky. 2015. Usando avaliações comutativas para comparar a compreensão conceitual em programas baseados em blocos e em texto. Em *Anais da Décima Primeira Conferência Internacional Anual sobre Pesquisa em Educação Computacional Internacional (ICER '15)*. ACM, Nova York, NY, EUA, 101–110. <https://doi.org/10.1145/2787622.2787721>
- [56] Susan Wiedenbeck, Deborah Labelle e Vennila NR Kain. 2004. Fatores que afetam os resultados do curso em programação introdutória. Em 16º Workshop Anual do Grupo de Interesse em Psicologia da Programação.
- [57] Brenda Cantwell Wilson e Sharon Shrock. 2001. Contribuindo para o sucesso em um curso introdutório de ciência da computação: um estudo de doze fatores. Em *Anais do Trigésimo Segundo Simpósio Técnico SIGCSE sobre Educação em Ciência da Computação (SIGCSE '01)*. ACM, Nova York, NY, EUA, 184–188. <https://doi.org/10.1145/364447.364581>
- [58] Benjamin Xie, Matthew J. Davidson, Min Li e Andrew J. Ko. 2019. Uma avaliação da Teoria de Resposta ao Item de uma avaliação de conhecimento CS1 independente de idioma. Em *Anais do 50º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (SIGCSE '19)*. ACM, Nova York, NY, EUA, 699–705. <https://doi.org/10.1145/3287324.3287370>
- [59] Benjamin Xie, Dastyni Loksa, Greg L. Nelson, Matthew J. Davidson, Dongsheng Dong, Harrison Kwik, Alex Hui Tan, Leanne Hwa, Min Li e Andrew J. Ko. 2019. Uma teoria de instrução para habilidades introdutórias de programação. *Computer Science Education* 29, 2-3 (2019), 205–253. <https://doi.org/10.1080/08993408.2019.1565235> arXiv:<https://doi.org/10.1080/08993408.2019.1565235>
- [60] Barry J Zimmerman. 2002. Tornando-se um aprendiz autorregulado: uma visão geral. *Teoria na prática* 41, 2 (2002), 64–70. <https://doi.org/10.1207/s15430421tip4102>